# Guion de la presentación

## Qué decir en cada slide · Entregable 4 — CI/CD

Sosa · Lapetina
Programación Avanzada — Universidad de Montevideo, 2026

## Parte 0 — Entender de qué va esto antes de hablar

Leé esta parte una vez, tranquila. No es para decirla: es para que cuando te pregunten algo fuera del guion, sepas de dónde agarrarte.

### ¿Qué es CI/CD y por qué existe?

Imaginate un equipo de cinco personas tocando el mismo código. Antes de que existiera esto, el procedimiento era: cada uno programaba por su lado, una vez por mes alguien juntaba todo, pasaba tres días arreglando los choques, probaba a mano lo que se acordaba de probar, y después copiaba los archivos al servidor. Si algo se rompía, se enteraban los usuarios.

**CI (integración continua)** es la respuesta a la primera mitad del problema: cada cambio se integra al código común enseguida, y una máquina lo compila y corre todas las pruebas automáticamente. Si rompiste algo, te enterás en minutos, no en un mes.

**CD (despliegue continuo)** es la segunda mitad: si las pruebas pasaron, esa misma máquina publica la versión nueva en el servidor, sola, siempre del mismo modo.

La idea de fondo es simple: **sacarle a las personas las tareas que hacen mal por aburrimiento** (acordarse de correr todas las pruebas, copiar los archivos correctos al lugar correcto) y dárselas a una máquina que las hace idénticas siempre.

### ¿Qué pide exactamente el entregable?

La letra pide construir una aplicación web chica en Java con Spring, con al menos dos módulos, y después —y acá está el verdadero trabajo— montarle todo el proceso de CI/CD alrededor:

1. Pruebas automáticas: unitarias de cada módulo y de integración del sistema entero.
2. Un pipeline de Jenkins **declarativo** y **versionado junto al código**.
3. Que ese pipeline **se dispare solo** cada vez que alguien hace push a GitHub.
4. Que despliegue en un **ambiente de producción simulado**.
5. En la defensa: mostrar un caso en que todo sale bien y otro en que las pruebas fallan.

La letra aclara en negrita que **no se evalúa la funcionalidad ni la apariencia de la aplicación**. Por eso nuestra app es deliberadamente chica.

## ¿Qué significan las palabras que vas a decir?

| Palabra | Qué es, en una línea |
|---|---|
| **Jenkins** | Un servidor que vigila el repositorio y ejecuta tareas solo cuando detecta cambios |
| **Pipeline** | La lista ordenada de pasos que Jenkins ejecuta: compilar, probar, desplegar… |
| **Jenkinsfile** | El archivo donde está escrito ese pipeline, guardado dentro del propio repositorio |
| **Pipeline as code** | Justamente eso: que el proceso viva en el repo y no en la configuración del servidor |
| **Etapa (stage)** | Cada paso del pipeline; en el tablero se ve como una columna verde o roja |
| **Prueba unitaria** | Prueba una pieza aislada, sin levantar la aplicación. Rápida |
| **Prueba de integración** | Levanta la aplicación entera y la usa por HTTP, como lo haría un usuario |
| **Docker / contenedor** | Una cajita aislada con la aplicación y todo lo que necesita adentro |
| **Imagen** | El molde del que salen los contenedores; la nuestra se etiqueta con el número de build |
| **Volumen** | Una carpeta de Docker que vive fuera del contenedor, así los datos no se pierden |
| **Smoke test** | Una verificación mínima después de desplegar: ¿la app responde o no? |
| **Rollback** | Volver a la versión anterior cuando la nueva falla |
| **Webhook** | Un aviso que GitHub le manda a Jenkins en el instante del push |
| **Polling** | Lo contrario: Jenkins le pregunta a GitHub cada tanto si hubo cambios |

## El mapa mental de toda la demo

Si te perdés en cualquier momento, volvé a esta frase:

> Hago un cambio → push a GitHub → Jenkins se entera solo → prueba → si pasa, despliega; si no pasa, frena y producción queda intacta.

Todo lo que vas a mostrar es una variación de esa frase.

### Las tres direcciones que vas a abrir

| Dirección | Qué es | Quién la levanta |
|---|---|---|
| `localhost:8080` | Jenkins, el tablero del pipeline | el contenedor de Jenkins |
| `localhost:8090` | La app en desarrollo, en tu máquina | vos, con `./scripts/correr-dev.sh` |
| `localhost:8091` | La app en producción simulada | el pipeline, automáticamente |

La diferencia que importa: **el 8090 lo controlás vos, el 8091 lo controla el pipeline.**

---

## Parte 1 — Antes de entrar al salón

Diez minutos antes, con calma:

```
# 1. Abrir Docker Desktop y esperar a que el ícono deje de moverse

# 2. Levantar Jenkins
cd "/Users/macbookpro/Desktop/programacion avanzada/Entregable4SosaLapetina"
./scripts/levantar-jenkins.sh

# 3. Verificar que producción responde
./scripts/estado.sh
```

Entrá a Jenkins y **dispará un build con Build Now antes de empezar**. No porque vaya a fallar: el primer build después de un reinicio tarda más mientras Maven acomoda sus cosas, y no querés ese silencio en vivo.

**Pestañas abiertas, en este orden:**

1. La presentación
2. `http://localhost:8080/job/catalogo-videojuegos/` — Jenkins
3. `http://localhost:8091` — producción
4. El repositorio en GitHub
5. El editor con el `Jenkinsfile` abierto
6. Una terminal, ya parada en la carpeta del proyecto

**El reparto entre las dos está en la Parte 1.5**, con los pases de posta y las reglas del dúo.

---

## Parte 1.5 — El reparto entre las dos

### Cómo queda

Miranda maneja la computadora de punta a punta (el proyecto está en su máquina) y es quien hace las tres demos en vivo. Emi lleva la parte explicativa. El reparto de la voz queda así:

| Slides | Quién habla | Qué es |
|---|---|---|
| 1 a 6 | **Emi** | Portada, el encargo, la aplicación, la arquitectura, la infraestructura y las pruebas |
| 7 a 11 | **Miranda** | El pipeline, los disparadores, las dos demos en vivo y el error de Maven |
| 12 | **Emi** | Las decisiones tomadas |
| 13 | **Miranda** | El cierre |

Unos 10 minutos Miranda y 8 y medio Emi. Emi abre, Miranda cierra.

El criterio detrás del reparto no es solo el tiempo, es **repartir la dificultad**: Miranda carga con lo que hay que ejecutar en vivo y con la única slide que pide entender algo complicado (la 11, el error de Maven), y Emi con lo descriptivo, que es contar lo que está en pantalla sin nada que pueda fallar.

### Las tres demos son de Miranda

| Momento | Slide | Dura |
|---|---|---|
| La aplicación funcionando: agregar un juego, filtrar por género | 3 | ≈ 2 min |
| Caso 1: un push que sale bien y llega a producción | 9 | ≈ 3 min |
| Caso 2: un push que rompe las pruebas y no llega | 10 | ≈ 3 min |

La de la slide 3 cae dentro del bloque de Emi: Emi explica los dos módulos mientras Miranda hace clic en el navegador. Las otras dos son enteramente de Miranda, slide y demo.

### Lo que hay que cuidar

**Miranda narra sus demos, no las tipea en silencio.** Es la diferencia entre que la demo cuente como exposición propia o que parezca que una presentó y la otra operó la computadora. Mientras el pipeline corre, la voz es de Miranda: *"ahí arrancó solo, están pasando las 30 unitarias, ahora las de integración…"*.

**Tener la compu no define quién habla.** Miranda maneja el teclado todo el tiempo — incluso durante las slides de Emi, cambiando de slide y abriendo pestañas — y eso está bien: quien habla no debería estar peleando con el trackpad.

### Los pases de posta

Que la transición sea una frase dicha, no un silencio mientras alguien se acomoda.

**Emi → Miranda, al terminar la slide 6:**

> *Hasta acá qué construimos y cómo lo probamos. Ahora Miranda les muestra el pipeline que corre todo esto solo.*

**Miranda → Emi, al terminar la slide 11:**

> *Eso fue lo que se nos rompió en el camino. Emi cuenta por qué tomamos algunas de las decisiones del proyecto.*

**Emi → Miranda, al terminar la slide 12:**

> *Y para cerrar, Miranda.*

### Las reglas del dúo

- **Una sola computadora**, la de Miranda, con las pestañas ya abiertas antes de empezar.
- **Mientras Jenkins corre, hablan las dos.** Son los 60 a 90 segundos más largos de la presentación si nadie dice nada. Emi puede acotar algo de las etapas que van pasando mientras Miranda sigue con la explicación.
- **Las preguntas las contesta quien dio esa parte.** Si no sabés, pasala en voz alta: *"eso lo armó Emi, que lo explica mejor"*. Es mucho mejor que improvisar.
- **Las dos tienen que poder contestar lo básico de todo.** El docente puede preguntarle a cualquiera cualquier cosa. La Parte 0 y la Parte 3 de este guion son justamente para eso: léanlas las dos.

### Cuánto tiene que preparar cada una

**Miranda, unos 30 minutos.** Leer su parte y, sobre todo, ensayar los comandos de las slides 9 y 10 dos veces para que salgan sin pensar. Lo único que tiene que *entender* de verdad es la slide 11, y son cuatro frases:

> El pipeline corre las pruebas en etapas separadas, llamando a los plugins de Maven por separado. Cuando se llama a un plugin suelto, Maven no corre el ciclo de vida completo, así que los módulos no llegan a generar su jar. Entonces clasificación sale a buscar el jar de catálogo afuera, a Maven Central, donde no está porque es nuestro. Se arregló cambiando `package` por `install`, que sí deja los jars en el repositorio local antes de que las pruebas los necesiten.

**Emi, unos 20 minutos.** Leer la Parte 0 de este guion y el texto de sus 7 slides. Son todas descriptivas: contar lo que está en pantalla. La más exigente es la 5, donde tiene que poder decir qué es un contenedor y por qué los datos van en un volumen aparte.

**Las dos:** la Parte 3, con las preguntas probables. El docente puede preguntarle cualquier cosa a cualquiera.

### Ensayo

Una pasada completa, con las dos demos de verdad y cronómetro. No para memorizar: para descubrir dónde se pisan, cuánto tarda el build en esta máquina y si los pases de posta suenan naturales. Una sola pasada completa cambia totalmente cómo sale.

## Parte 2 — Qué decir en cada slide

Lo que está en cursiva es la idea, no un texto para memorizar. Decilo con tus palabras: si suena leído, se nota.

### Slide 1 · Portada — 30 segundos · Emi

**Mostrar:** la portada.

*Buenas. Somos Sosa y Lapetina. Este es el Entregable 4, sobre integración y despliegue continuo. Construimos un catálogo de videojuegos, pero aclaramos de entrada que la aplicación es lo de menos: lo que armamos y lo que vamos a mostrar es el proceso que lleva ese código desde que lo escribimos hasta que está funcionando en producción, sin que nadie toque un botón.*

> No te disculpes por la app (“es simple”, “no llegamos a…”). Fue una decisión, no una limitación. Decilo así.

### Slide 2 · El encargo — 1 minuto · Emi

**Mostrar:** las cuatro tarjetas.

*La letra pedía cuatro cosas: una app con al menos dos módulos, pruebas automáticas de cada módulo y de integración, un pipeline de Jenkins declarativo versionado junto al código y que se dispare con cada cambio, y un ambiente de producción simulado donde desplegar.*

*Y aclaraba en negrita que no se evalúa la funcionalidad ni la apariencia. Nosotras tomamos eso en serio: elegimos el dominio más chico de los tres propuestos para poder poner el esfuerzo en la automatización, y para tener menos cosas que puedan fallar hoy acá.*

### Slide 3 · La aplicación — 1 minuto · Emi, con Miranda en el navegador

**Mostrar:** las dos tarjetas de módulos y, abajo, el recuadro del punto de integración.

*La app es un catálogo de videojuegos con dos módulos. El primero, catálogo, se encarga del alta, la baja y la consulta, con sus reglas: el título es obligatorio y no se puede repetir, el año tiene que estar*

*entre 1958 y hoy. El segundo, clasificación, maneja los géneros: los normaliza, filtra el catálogo y saca estadísticas.*

*Lo importante es dónde se tocan. Cuando das de alta un juego, el género que escribiste pasa primero por clasificación —si escribís "rpg" en minúscula, te lo convierte en "RPG"— y recién ahí catálogo lo guarda. Ese camino, el que atraviesa los dos módulos, es justamente lo que prueban las pruebas de integración.*

> **Acá cambiá al navegador, a `localhost:8091`.** Agregá un juego, filtrá por género, señalá la etiqueta PRODUCCIÓN arriba a la derecha. **Dos minutos como máximo**, y volvé a las slides. Es la parte que menos puntos da.

### Slide 4 · Arquitectura — 1 minuto · Emi

**Mostrar:** la pila de los tres módulos con las flechas.

*Es un proyecto Maven multi-módulo. Las dependencias van en una sola dirección: la aplicación web conoce a los dos módulos, clasificación conoce a catálogo, y catálogo no conoce a nadie.*

*Lo hicimos así, y no con paquetes dentro de un solo proyecto, por dos razones. Una, que las dependencias quedan explícitas: si alguien intenta invertirlas, Maven no lo deja. Y dos, que cada módulo tiene su propia suite de pruebas, así que cuando algo falla el log del pipeline dice exactamente qué módulo fue.*

*Una cosa más: el repositorio trae el Maven Wrapper, así que ni Jenkins ni nosotras instalamos Maven. Todos construimos con la misma versión exacta.*

### Slide 5 · Infraestructura — 1 minuto · Emi

**Mostrar:** el diagrama de cuatro cajas.

*Esto es lo que corre cuando todo está funcionando. A la izquierda nuestra máquina con la app en desarrollo. Hacemos push a GitHub. Jenkins, que corre en un contenedor, se entera y ejecuta el pipeline. Y si todo pasa, levanta el contenedor de producción.*

*Dos detalles que nos costaron. El primero: Jenkins no lleva un Docker adentro, usa el del sistema montando su socket. Es un patrón que se llama Docker outside of Docker, y es más simple y más estable que meter un Docker dentro de otro.*

*El segundo: cada despliegue destruye y vuelve a crear el contenedor de producción. Si los datos vivieran adentro, se perderían en cada deploy. Por eso están en un volumen aparte, y los juegos cargados sobreviven. Eso lo van a ver en la demo.*

### Slide 6 · Pruebas — 1 minuto · Emi

**Mostrar:** los dos números grandes, 30 y 10.

*Tenemos 30 pruebas unitarias y 10 de integración. Las unitarias no levantan Spring: usan un repositorio en memoria que escribimos nosotras como doble de prueba, y corren en milisegundos. Las de integración sí levantan la aplicación entera en un puerto real y la consultan por HTTP, como lo haría un usuario.*

*Las separamos usando la convención de Maven: los archivos que terminan en Test los corre surefire, los que terminan en IT los corre failsafe. Es solamente el nombre del archivo, no cuesta nada, y a cambio el pipeline las corre en etapas distintas. Cuando algo falla, el tablero dice al toque si se rompió la lógica de un módulo o la interacción entre módulos.*

### Slide 7 · El pipeline — 1 minuto y medio · Miranda

**Mostrar:** la tabla de ocho etapas. **Después abrí el `Jenkinsfile` en el editor.**

*Estas son las ocho etapas. Clona, compila, corre las unitarias, corre las de integración, guarda el jar, construye la imagen de Docker etiquetada con el número de build, despliega, y verifica que la versión nueva responda.*

*Miren la última columna, que es la importante: hasta la etapa seis, si algo falla producción ni se entera. Recién en la siete tocamos producción, y si el smoke test de la ocho falla, volvemos atrás solos.*

*Y todo esto está en el Jenkinsfile, acá, en la raíz del repositorio. Eso es lo que significa pipeline as code: el proceso de construcción viaja con el código, y cualquier cambio al proceso queda en el historial de Git como cualquier otro cambio.*

> No leas el Jenkinsfile línea por línea. Mostralo, señalá el bloque `stages` y el bloque `post`, y volvé.

### Slide 8 · Disparadores — 45 segundos · Miranda

**Mostrar:** el bloque de código con `githubPush()` y `pollSCM`.

*Nadie aprieta un botón. Dejamos configurados los dos mecanismos. El webhook es el ideal: GitHub le avisa a Jenkins en el instante del push. Pero nuestro Jenkins corre en localhost, y GitHub no puede entrar a una máquina que no es visible desde internet. Por eso además tenemos polling: Jenkins le pregunta a GitHub cada dos minutos si hubo cambios. Eso es lo que van a ver funcionando ahora.*

### Slide 9 · Caso 1 — 3 minutos (demo en vivo) · Miranda

**Mostrar:** la slide primero, después la terminal, después Jenkins, después producción.

*Vamos al primer caso: una integración que sale bien. Voy a hacer un cambio chiquito y visible.*

En el editor, en `modulo-clasificacion/.../Genero.java`, agregá una línea:

```
PLATAFORMAS("Plataformas"),
```

En la terminal:

```
git add -A
git commit -m "Agrega el genero Plataformas"
git push
```

*Ya está. No toco nada más.*

**Pasá a Jenkins y recargá hasta que arranque** (menos de dos minutos). Mientras corre:

*Ahí arrancó solo. Pueden ver las etapas pasando: compiló, están corriendo las 30 unitarias… ahora las 10 de integración… construyó la imagen, la desplegó, y el smoke test confirma que responde.*

**Cuando termine, pasá a `localhost:8091` y recargá.**

*Acá está el género nuevo en el desplegable. Y fíjense en algo: los juegos que había cargados siguen estando. El contenedor se destruyó y se volvió a crear, pero los datos viven en el volumen.*

> Si el polling tarda, no te quedes en silencio: contá que está configurado cada dos minutos y que en un entorno real el webhook lo haría instantáneo. O apretá Build Now y decí que lo estás forzando para no hacerlos esperar.

### Slide 10 · Caso 2 — 3 minutos (demo en vivo) · Miranda

**Mostrar:** la slide, después el editor, después Jenkins, después producción.

*Ahora el caso interesante: qué pasa cuando alguien sube código roto.*

En `ServicioClasificacion.java`, cambiá el método `normalizar` por:

```
public String normalizar(String generoLibre) {
    return generoLibre;
}
```

*Esto parece una simplificación inocente, pero rompe la normalización de géneros.*

```
git commit -am "Simplifica la normalizacion de generos"
git push
```

**En Jenkins, cuando falle:**

*Compiló bien, porque es código Java válido. Pero miren: la etapa de pruebas unitarias está en rojo. Y acá dice exactamente qué pasó: la prueba `normalizaGenero` esperaba "RPG" y recibió "rpg".*

*Y ahora lo más importante de toda la presentación: miren las etapas de abajo. Imagen Docker, skipped. Desplegar, skipped. Smoke test, skipped. El pipeline frenó y nunca llegó a producción.*

**Pasá a `localhost:8091` y recargá.**

*Producción sigue funcionando con la versión anterior. El error quedó encerrado entre el push y el despliegue, y nadie tuvo que acordarse de nada.*

*¿Cómo se sale de acá? Se corrige o se revierte, se vuelve a pushear, y el pipeline corre de nuevo.*

```
git revert --no-edit HEAD
git push
```

*Y cuando pase en verde, ahí sí llega a producción.*

### Slide 11 · Errores encontrados — 1 minuto y medio · Miranda

**Mostrar:** el recuadro grande del error de Maven.

*Nos pidieron documentar los errores del pipeline, y tuvimos uno que vale la pena contar, porque es el que más nos hizo entender cómo funciona Maven.*

*El proyecto compilaba y pasaba todas las pruebas en nuestras máquinas, pero el pipeline fallaba diciendo que no encontraba el jar de módulo-catálogo en Maven Central. Que obviamente no está, porque es nuestro.*

*Lo que pasaba es esto: el pipeline corre las pruebas en etapas separadas invocando los plugins sueltos, y cuando invocás un goal suelto Maven no ejecuta el ciclo de vida completo. Entonces los módulos del reactor no tienen un jar para ofrecerse entre sí, y clasificación sale a buscar el de catálogo afuera. Localmente nunca lo vimos porque nosotras corríamos el ciclo completo con un solo comando.*

*Lo resolvimos cambiando la etapa de compilación de `package` a `install`, para que los jars queden en el repositorio local antes de que las pruebas los necesiten.*

> Esta es la slide que más demuestra que entendieron lo que hicieron. Contala sin apuro. Los otros tres errores están abajo y en la sección 6 del informe: solo mencionalos.

### Slide 12 · Decisiones — 1 minuto · Emi

**Mostrar:** la tabla.

> No la leas entera. Elegí dos o tres filas.

*Algunas decisiones y por qué. Persistimos en un archivo JSON y no en una base de datos: una base habría agregado otro contenedor, credenciales y migraciones al ambiente de producción sin aportar nada al objetivo del entregable, que es el proceso. El archivo está en un volumen, que es lo que demuestra lo que había que demostrar.*

*Y producción en Docker en vez de una carpeta local porque nos da tres cosas: el despliegue es reproducible en cualquier máquina, cada versión queda etiquetada con su número de build, y por lo tanto el rollback es real, no un copiar y pegar.*

### Slide 13 · Cierre — 30 segundos · Miranda

*Para cerrar: lo que nos llevamos es que escribir código que funciona y tener un proceso que garantice que lo que llega a producción funciona son dos cosas distintas. La app es chica, pero ningún cambio llega al ambiente de producción sin pasar por compilación, cuarenta pruebas y una verificación de salud, sin que nadie se acuerde de nada.*

*Lo próximo que agregaríamos es un ambiente de UAT con una aprobación manual antes de promover a producción.*

*Gracias. ¿Preguntas?*

---

## Parte 3 — Preguntas probables

**¿Por qué el pipeline está en el repositorio y no configurado en Jenkins?** Porque así el proceso de construcción versiona junto con el código. Si mañana una rama necesita una etapa distinta, la trae en su propio Jenkinsfile, y cualquier cambio al proceso queda en el historial de Git y se puede revisar como cualquier otro cambio.

**¿Cómo sabe Jenkins que hubo un cambio?** Dos mecanismos: el webhook de GitHub y, como respaldo porque Jenkins corre en localhost y GitHub no lo alcanza, polling cada dos minutos. En el build que mostramos, Jenkins registra el motivo textualmente: *Started by an SCM change*.

**¿Qué pasa si falla el despliegue y no las pruebas?** El bloque `post { failure }` detecta que había una imagen anterior desplegada y hace rollback. Cada build etiqueta su imagen con su número, así que volver atrás es un `docker run`.

**¿Por qué archivo y no base de datos?** Porque la persistencia no es el objeto del entregable y una base agregaría infraestructura sin aportar al proceso. Lo que había que demostrar —que los datos sobreviven al despliegue— se demuestra igual con el volumen.

**¿Dónde corre Maven?** No se instala en ningún lado: el repositorio trae el Maven Wrapper, así que Jenkins y nosotras usamos exactamente la misma versión, la 3.9.9.

**¿Por qué Docker outside of Docker y no Docker in Docker?** Porque anidar un daemon dentro de otro es más frágil y más pesado, y no lo necesitábamos: alcanzaba con que Jenkins pudiera hablarle al Docker del sistema.

**¿Cuánto tarda el pipeline?** Alrededor de un minuto y medio con la caché caliente. El primero después de reiniciar tarda más porque Maven baja las dependencias; por eso dejamos el repositorio local de Maven en el volumen persistente de Jenkins.

**¿Probaron que el rollback funciona?** El caso que mostramos es el de fallo antes del despliegue, donde producción ni se toca. El rollback cubre el otro escenario, cuando el contenedor nuevo arranca pero no responde al smoke test.

---

## Parte 4 — Si algo sale mal en vivo

| Qué pasa | Qué hacer |
|---|---|
| Jenkins no abre en 8080 | ¿Docker Desktop está abierto? `./scripts/levantar-jenkins.sh` de nuevo y esperá un minuto |
| El polling no arranca el build | Apretá **Build Now** y explicá que lo forzás para no hacerlos esperar |
| Producción no responde en 8091 | `./scripts/estado.sh` para ver el estado; si el contenedor no está, `./scripts/desplegar-local.sh` |
| El push pide contraseña | Es el token, no la contraseña de GitHub |
| Se cae internet | Mostrá el Anexo A del informe: tiene la salida real de los dos casos |

Y la regla general: si algo se traba, **seguí hablando**. Explicá qué debería estar pasando mientras lo arreglás. El silencio se nota mucho más que el problema.